



BigData

**An-Najah National University
Engineering and Information Technology**

**Final Project
Opportunity Matcher Project**

**Created by:
Aprar Ismail
Saja Dwaikat
Adham Iahlooh
Ahmad Istatieh**

**Under Supervision:
Dr. Hamed Abdelhaq**

Academic Year: 2025 / 2026

Opportunity Matcher

1. Project Overview

The Opportunity Matcher platform is a smart data-driven system designed to match students with suitable opportunities such as internships, academic projects, or job positions. The matching process relies on multiple attributes, including students' skills, completed courses, and GPA, in order to provide accurate and relevant recommendations.

Given the potentially large number of student profiles and opportunity descriptions, traditional matching approaches become inefficient and computationally expensive. Therefore, this project adopts a Big Data architecture that enables scalable data ingestion, real-time processing, and efficient similarity computation.

The system is implemented using Scala and integrates several Big Data technologies. Apache Kafka is used for streaming student data, Apache Spark for distributed data processing, MongoDB for flexible and scalable storage, and Akka HTTP to expose backend services as APIs.

The frontend is built using HTML, CSS, and JavaScript, allowing users to upload opportunity data and view matching results. The system supports uploading opportunity descriptions in multiple file formats, including PDF, DOC, DOCX, and TXT, which are automatically processed and converted into a structured representation suitable for similarity matching.

For similarity matching, the system uses Locality Sensitive Hashing (LSH) to efficiently estimate similarity between student profiles and opportunities.

2. System Architecture

The system follows a modular Big Data architecture composed of multiple loosely coupled components. Each component has a clear responsibility, enabling scalability, maintainability, and efficient data flow.

At a high level, the architecture consists of:

- A **Kafka Producer** for data ingestion
- A **Spark-based Consumer pipeline** for streaming processing
- A **MongoDB database** for storage
- A **Matching engine** based on LSH
- An **API layer** using Akka HTTP
- A **Frontend interface** for user interaction

This architecture allows continuous ingestion of student data, real-time processing, and fast retrieval of matching results.

3. Implementation Details

3.1 Data Ingestion – Kafka Producer

Student data is ingested using a Kafka Producer implemented in `KafkaProducerData.scala`. The producer reads student profiles from a CSV file and sends them as JSON messages to a Kafka topic.

Kafka is used to support **continuous and scalable data ingestion**, allowing the system to handle large volumes of data without tight coupling between producers and consumers. Data is sent in batches of 500 records, and the system tracks the last processed record to avoid duplicate ingestion.

3.2 Streaming Processing – Spark Consumer

The Kafka Consumer is implemented using **Spark Structured Streaming**. It reads messages from the Kafka topic, parses JSON data into Spark DataFrames, and performs data cleaning and transformation.

Unnecessary attributes such as gender, address, and birth date are removed, and date fields are converted into appropriate formats. Spark enables distributed processing, making the pipeline scalable and suitable for large datasets.

3.3 Data Storage – MongoDB

Processed data is stored in **MongoDB**, a NoSQL database chosen for its schema flexibility and scalability. MongoDB supports storing semi-structured data such as arrays of skills and courses, which are essential for similarity matching.

The system uses **upsert operations**, ensuring that existing student records are updated while new records are inserted when necessary. This approach avoids duplication and maintains data consistency.

3.4 API Layer – Akka HTTP

The API layer is implemented using **Akka HTTP** and serves as the communication bridge between the frontend and the backend logic. The API exposes an endpoint (`match_students`) that accepts opportunity data in JSON format and returns the top matching students.

Akka HTTP is used for its asynchronous and non-blocking nature, which improves performance and responsiveness under concurrent requests.

4. Probabilistic Data Structure: Locality Sensitive Hashing (LSH)

4.1 Motivation for Probabilistic Matching

Exact similarity matching between all students and all opportunities requires pairwise comparisons, resulting in high computational complexity ($O(N^2)$). Such an approach is not suitable for large-scale data.

To address this limitation, the system uses **Locality Sensitive Hashing (LSH)**, a probabilistic technique that enables approximate similarity matching with significantly lower computational cost.

4.2 LSH Implementation

Student profiles and opportunities are represented as feature vectors derived from skills, courses, and majors. Text data is converted into numerical vectors using **HashingTF**.

Similarity is then computed using **MinHashLSH**, which approximates **Jaccard similarity** between vectors.

The system uses `approxNearestNeighbors` to retrieve the top 5 most similar students for a given opportunity.

4.3 Performance Note

LSH is faster than exact similarity matching because it reduces the number of comparisons by hashing similar vectors into the same buckets. Instead of comparing an opportunity with all student profiles, the system only compares candidates within the same hash buckets. This significantly reduces computation time and makes the solution scalable for large datasets.

5. Data Flow Overview

1. The **Producer** reads student data from a CSV file and sends it to Kafka as JSON messages.
2. The **Consumer pipeline** (Kafka Consumer + Spark) processes and cleans the data.
3. Cleaned data is stored in **MongoDB**.
4. The **API Server** receives opportunity requests and triggers the matching process.
5. The **Frontend** sends requests and displays matching results to users.

6. Results

The system successfully returns relevant student matches for a given opportunity based on similarity of skills, courses, and major.

Example Input:

An opportunity description uploaded as a PDF or DOCX file containing required skills and courses.

Example Output:

The system returns the top 5 most similar students, including their name, major, skills, and similarity score.

These results demonstrate the effectiveness of the LSH-based matching approach in retrieving relevant candidates efficiently.

7. Challenges

Several challenges were encountered during development:

1. JSON Parsing Issues:

Differences in input formats required careful validation and error handling.

2. Kafka Configuration and Connectivity:

Initial issues with broker configuration and topic availability caused intermittent failures.

3. Streaming Stability:

Spark streaming jobs occasionally faced flakiness during restarts, requiring careful checkpointing and configuration tuning.

8. Conclusion

This project demonstrates a complete Big Data solution for matching students with opportunities using scalable data ingestion, distributed processing, and probabilistic similarity matching. By integrating Kafka, Spark, MongoDB, and LSH, the system efficiently handles large datasets and provides fast and relevant matching results.

9. Future Work

Future improvements may include:

- Incorporating weighted similarity scoring
- Adding real-time dashboards
- Evaluating matching accuracy using quantitative metrics
- Supporting additional data sources